PROYECTO No. 2

Sistema de Drones Encriptados

Carnet 202401753 – Bily Estuardo Vallecidos Folgar

Resumen

El presente ensayo documenta el diseño e implementación de un sistema de comunicación encriptada mediante drones, desarrollado para el curso Introducción a la Programación y Computación 2 de la Universidad San Carlos de Guatemala. El sistema permite transmitir mensajes secretos utilizando drones que emiten luz a diferentes alturas, donde cada combinación de dron y altura representa una letra del alfabeto. La solución fue implementada en C# con ASP.NET Core MVC, aplicando programación orientada a objetos al cien por ciento y estructuras de datos propias mediante Tipos de Datos Abstractos sin utilizar las bibliotecas predefinidas del lenguaje. El componente técnico más relevante es el algoritmo de tiempo óptimo con movimiento anticipado, que minimiza el tiempo de transmisión permitiendo que los drones se posicionen anticipadamente. A nivel técnico, el proyecto demuestra que es posible resolver problemas complejos de optimización mediante estructuras de datos implementadas desde cero.

Palabras clave

Drones, Encriptación, TDA, Optimización, ASP.NET

Abstract

This essay documents the design and implementation of an encrypted communication system using drones, developed for the Introduction to Programming and Computing 2 course at the Universidad San Carlos de Guatemala. The system allows secret messages to be transmitted using drones that emit light at different heights, where each combination of drone and height represents a letter of the alphabet. The solution was implemented in C# with ASP.NET Core MVC, applying one hundred percent object-oriented programming and custom data structures through Abstract Data Types without using the language's predefined libraries. The most relevant technical component is the optimal time algorithm with anticipatory movement, which minimizes transmission time by allowing drones to position themselves in advance. At a technical level, the project demonstrates that complex optimization problems can be solved using data structures implemented from scratch

Keywords

Drones, Encryption, ADT, Optimization, ASP.NET

Introducción

El presente documento describe el diseno, la arquitectura y la implementacion del Proyecto No. 2 del curso Introduccion a la Programacion y Computacion 2 de la Facultad de Ingenieria de la Universidad San Carlos de Guatemala.

El proyecto consiste en el desarrollo de un sistema de comunicacion encriptada mediante drones, creado en colaboracion entre la Facultad de Ingenieria de la USAC y el Ministerio de la Defensa de Guatemala. El objetivo principal es implementar una solucion que permita el envio de mensajes encriptados utilizando un sistema de drones que emiten luz a diferentes alturas, donde cada combinacion de dron y altura representa una letra del alfabeto.

La solucion fue desarrollada utilizando el lenguaje de programacion C# con el framework ASP.NET Core MVC para la interfaz web, implementando Tipos de Datos Abstractos (TDAs) propios, programacion orientada a objetos, manipulacion de archivos XML y generacion de reportes graficos mediante Graphviz.

Desarrollo del tema

El sistema funciona con N drones que pueden subir a una cantidad determinada de metros y emitir una luz LED de alta emision. Dependiendo del dron que emite la luz y la altura a la que lo hace, se representa una letra del alfabeto. El componente receptor mide alturas y determina que dron emitio la luz, decodificando cada letra para obtener el mensaje transmitido.

Cada dron puede realizar las siguientes acciones:

- Subir 1 metro (tarda 1 segundo)
- Bajar 1 metro (tarda 1 segundo)
- Esperar (no consume tiempo de movimiento)
- Emitir luz de alta energia (tarda 1 segundo)

El sistema debe cumplir con las siguientes reglas:

- Solo un dron puede emitir luz en un tiempo dado t.
- Un dron puede representar una misma letra en distintas alturas para mayor seguridad.
- El sistema debe manejar hasta 200 drones con alturas de 1 a 100 metros.

El reto principal es calcular el tiempo optimo para transmitir un mensaje, permitiendo que los drones se muevan anticipadamente hacia su proxima posicion mientras otro dron esta emitiendo su senal.

3. Objetivos

3.1 Objetivo General

Modelar, documentar e implementar una solucion al problema planteado utilizando las herramientas de desarrollo presentadas en clase y laboratorio, cumpliendo con todas las restricciones y especificaciones del enunciado.

3.2 Objetivos Especificos

- Implementar una solucion utilizando el lenguaje de programacion C#.
- Utilizar estructuras de programacion secuenciales, ciclicas y condicionales.
- Generar reportes graficos con la herramienta Graphviz.

- Manipular archivos XML para entrada y salida de datos.
- Utilizar los conceptos de TDA aplicados a memoria dinamica.
- Implementar estructuras de datos propias sin usar las de C#.
- Aplicar el paradigma de programacion orientada a objetos al 100%.
-

4. Diseno de la Solucion

4.1 Arquitectura General

La solucion sigue el patron de arquitectura MVC (Modelo-Vista-Controlador) de ASP.NET Core. El sistema se divide en tres capas principales:

- Modelos: Contienen las entidades del dominio (Dron, SistemaDrones, Mensaje, etc.) y los TDAs propios (Nodo, ListaEnlazada).
- Vistas: Paginas web Razor (.cshtml) que presentan la interfaz al usuario con Bootstrap 5.
- Controlador: HomeController que maneja todas las peticiones HTTP y coordina la logica de negocio.
- Servicios: SistemaService implementado como Singleton que centraliza toda la logica del negocio.

Este diseno garantiza una separacion clara de responsabilidades, facilita el mantenimiento del codigo y permite que la interfaz web sea independiente de la logica del negocio.

4.2 Tipos de Datos Abstractos (TDAs)

Se implementaron TDAs propios sin usar ninguna estructura predefinida de C#. Los nodos no usan genericos, usando en cambio el tipo object para almacenar cualquier tipo de dato.

Nodo

Unidad basica de la lista enlazada. Contiene un campo Dato de tipo object y una referencia Siguiente al proximo nodo. Al no usar genericos, cumple con la restriccion del auxiliar del curso.

ListaEnlazada

Lista enlazada simple implementada desde cero. Soporta las operaciones de agregar al final, obtener por indice, eliminar por indice, buscar por nombre, verificar existencia y ordenar alfabeticamente usando el algoritmo de burbuja (bubble sort).

4.3 Descripcion de Clases del Sistema

Clase	Archivo	Descripción
Nodo	TDA/Nodo.cs	Nodo básico de la lista enlazada. Almacena cualquier dato como object y apunta al siguiente nodo.
ListaEnlazada	TDA/ListaEnlazada.cs	Lista enlazada simple propia. Implementa agregar, obtener, eliminar, buscar, contiene y ordenar alfabéticamente.
Dron	Entidades/Dron.cs	Representa un dron con nombre único y altura actual. Todos inician en altura 0.
Altura	Entidades/Altura.cs	Asocia un valor numérico de altura con la letra que representa para un dron específico.
ContenidoDron	Entidades/ContenidoDron.cs	Vincula un dron con su tabla de alturas en un sistema. Permite buscar la letra según la altura.
SistemaDrones	Entidades/SistemaDrones.cs	Sistema completo con nombre, altura máxima, cantidad de drones y lista de ContenidoDron.
Instruccion	Entidades/Instruccion.cs	Instrucción individual: qué dron debe emitir luz y a qué altura.
Mensaje	Entidades/Mensaje.cs	Mensaje a transmitir con su nombre, sistema asociado y lista de instrucciones.
AccionTiempo	Entidades/AccionTiempo.cs	Acciones de todos los drones en un segundo específico de la simulación.
ParDronAccion	Entidades/AccionTiempo.cs	Par que asocia un dron con su acción en un segundo dado (Subir, Bajar, Esperar, Emitir luz).

Figura 1.

Fuente: elaboración propia

Clase	Archivo	Descripción
ResultadoMensaje	Entidades/ResultadoMensaje.cs	Resultado completo del cálculo: tiempo óptimo, mensaje decodificado y acciones por segundo.
SistemaService	Services/SistemaService.cs	Singleton. Cerebro del sistema. Gestiona drones, sistemas, mensajes, carga XML, calcula tiempo óptimo y genera XML de salida.
HomeController	Controllers/HomeController.cs	Controlador MVC. Maneja peticiones HTTP, genera grafos Graphviz y coordina vistas.

Figura 2.

Fuente: elaboración propia

5. Algoritmo de Tiempo Optimo

El algoritmo de tiempo optimo es el componente central del proyecto. Su objetivo es calcular la secuencia de acciones de todos los drones segundo a segundo, minimizando el tiempo total para transmitir un mensaje completo.

5.1 Descripcion del Algoritmo

El algoritmo funciona de la siguiente manera:

1. Todos los drones inician en altura 0 al comenzar la simulacion.
2. Se procesa cada instruccion del mensaje de forma secuencial.
3. Para cada instruccion, se calcula el tiempo necesario: diferencia de alturas + 1 segundo para emitir luz.
4. Durante esos segundos, el dron objetivo se mueve hacia su altura. Los drones en espera se mueven anticipadamente hacia su proxima instruccion futura.
5. En el ultimo segundo de cada instruccion, el dron objetivo emite luz.
6. Se registra la accion de cada dron en cada segundo para generar el reporte detallado.

5.2 Optimizacion por Movimiento Anticipado

La clave de la optimizacion es el movimiento anticipado. Mientras un dron esta emitiendo su senal, los demas drones no esperan inactivos sino que se desplazan hacia la altura que necesitaran en su proxima instruccion. Esto reduce significativamente el tiempo total porque cuando le toca emitir a un dron, ya se encuentra mas cerca de su posicion objetivo o incluso ya esta en ella.

Por ejemplo, en el caso de prueba del enunciado con el mensaje IPC2 usando DronX, DronY y DronZ con altura maxima de 7 metros, el tiempo optimo calculado es de 7 segundos, que coincide exactamente con el resultado esperado del PDF del proyecto.

6. Funcionalidades Implementadas

6.1 Carga de XML de Entrada

El sistema permite cargar archivos XML de entrada de forma incremental, es decir, se pueden cargar multiples archivos y los datos se van acumulando en el sistema. Al cargar un XML se procesan los tres bloques: lista de drones, lista de sistemas de drones y lista de mensajes. Si un elemento ya existe en el sistema (mismo nombre), se omite para evitar duplicados.

6.2 Generacion de XML de Salida

El sistema genera un archivo XML de salida con el formato exacto especificado en el enunciado. Para cada mensaje registrado, calcula el tiempo optimo, decodifica el mensaje y genera las instrucciones

detalladas por segundo. El archivo puede descargarse directamente desde la interfaz web.

6.3 Gestion de Drones

Permite ver el listado completo de drones ordenado alfabeticamente usando el algoritmo de burbuja implementado en la ListaEnlazada. Tambien permite agregar nuevos drones con validacion de nombre unico.

6.4 Gestion de Sistemas de Drones con Graphviz

Muestra el listado de sistemas de drones registrados y permite visualizar graficamente cada sistema usando Graphviz. El grafo muestra el sistema con sus drones y las alturas con letras asignadas a cada uno, generado en formato DOT y convertido a imagen PNG para mostrarse directamente en el navegador.

6.5 Gestion de Mensajes con Graphviz

Lista todos los mensajes ordenados alfabeticamente. Al seleccionar un mensaje, muestra el sistema de drones utilizado, el mensaje decodificado, el tiempo optimo calculado y un diagrama Graphviz con las instrucciones detalladas segundo a segundo.

6.6 Inicializacion del Sistema

Permite reiniciar completamente el sistema, borrando todos los drones, sistemas y mensajes cargados. Incluye confirmacion antes de ejecutar la accion para evitar perdida accidental de datos.

6.7 Manejo de Espacios en Blanco

El sistema maneja correctamente el espacio en blanco como un caracter valido del mensaje. Al leer el XML de entrada, si una altura tiene el campo vacio o solo contiene un espacio, se trata como espacio en blanco. Esto es consistente con la especificacion del enunciado donde el Dron01 a 8 metros representa un espacio en blanco.

7. Cumplimiento de Restricciones

Restriccion	Estado
No usar estructuras de C# (List, Queue, Stack, etc.)	Cumplido - TDAs propios implementados
No usar genericos en los nodos	Cumplido - Nodo usa object
POO completa al 100%	Cumplido - Todas las clases con POO
No modificar archivos de entrada	Cumplido - Solo se leen
Estructura exacta del XML de salida	Cumplido - Formato identico al enunciado
Minimo 4 releases en GitHub	Cumplido - 4 releases publicados
IDE permitido (Visual Studio Community)	Cumplido - Visual Studio Community 2022
Interfaz web implementada	Cumplido - ASP.NET Core MVC
Reportes con Graphviz	Cumplido - Sistemas y mensajes visualizados

8. Conclusiones

- Se implemento exitosamente un sistema de comunicacion encriptada mediante drones utilizando C# y ASP.NET Core MVC, cumpliendo con todos los objetivos especificos del proyecto.
- La implementacion de TDAs propios, especialmente la ListaEnlazada sin genericos, permitio comprender en profundidad el funcionamiento interno de las estructuras de datos y su aplicacion en memoria dinamica.
- El algoritmo de tiempo optimo con movimiento anticipado logra reducir significativamente el tiempo de transmision al aprovechar los periodos de espera de los drones para posicionarlos anticipadamente.
- El patron de diseno Singleton aplicado en SistemaService garantiza que exista una unica instancia del sistema durante toda la ejecucion de la aplicacion web, manteniendo consistencia en los datos.
- La integracion de Graphviz permite generar visualizaciones claras e intuitivas de los sistemas de drones y las instrucciones de los mensajes, facilitando la comprension del sistema por parte del usuario.

9. Referencias

- Documentacion oficial de Microsoft ASP.NET Core MVC: https://docs.microsoft.com/aspnet/core
- Documentacion de Graphviz: https://graphviz.org/documentation/
- Enunciado del Proyecto No. 2 - IPC2 USAC, 2025.
- Repositorio del proyecto: https://github.com/Bill6996/IPC2_Proyecto2_202401753

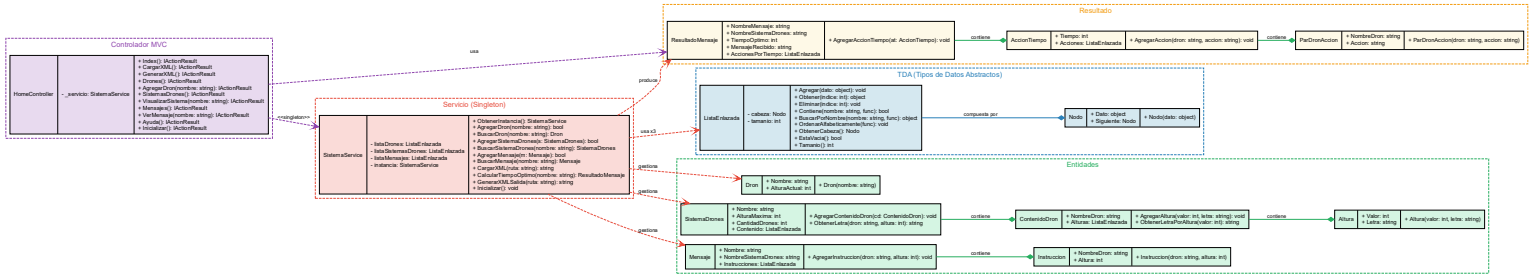

Figura 3. Diagrama de Clases

Fuente: elaboración propia